

计算机组成与结构

(2026春季)

量化研究方法

刘 铎

办公室: 信息大楼B822

邮箱: liuduo@cqu.edu.cn

计算机系统架构与量化研究方法 (Quantitative Approach)

通过使用工具来测试**设计思路及其权衡**，从而估计其对性能、功耗和成本的影响（这是一个迭代过程）。

包括：

- 分析性推理与基本设计原则
- 基本度量指标的方程
 - **成本、性能、功耗等**
- 不同层次的仿真
 - **系统级、ISA 级、微体系结构级、存储器级、RTL 级、门级、电路级**
- 代表典型工作负载的基准程序

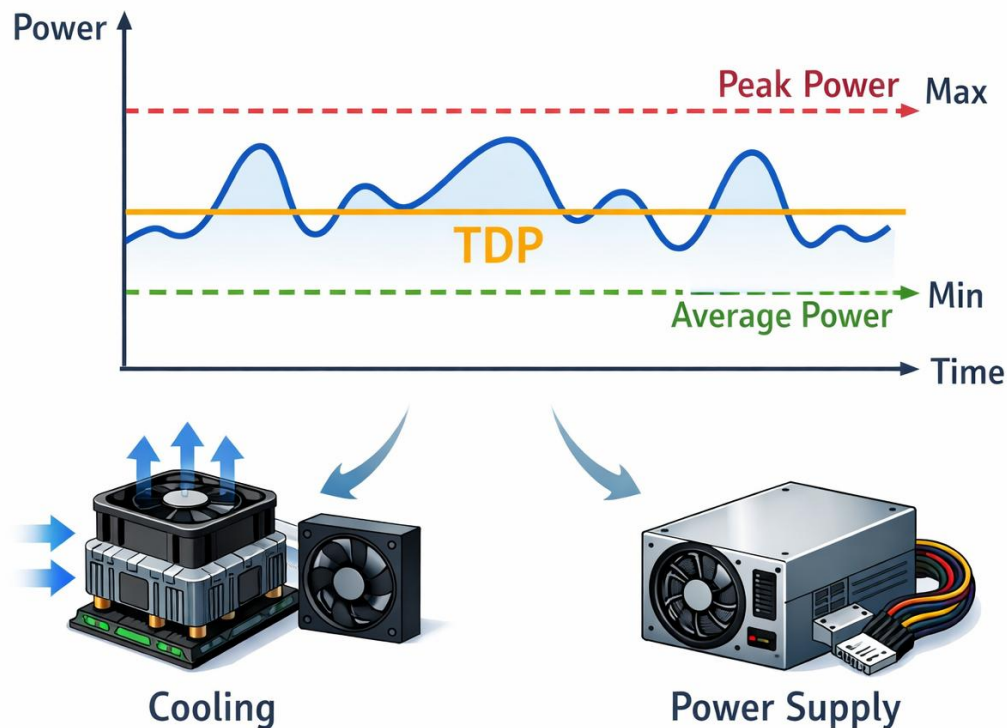
什么是热设计功耗 (TDP)

问题：有功率输入，就必须有功率输出（即散热）。

热设计功耗 (TDP)：

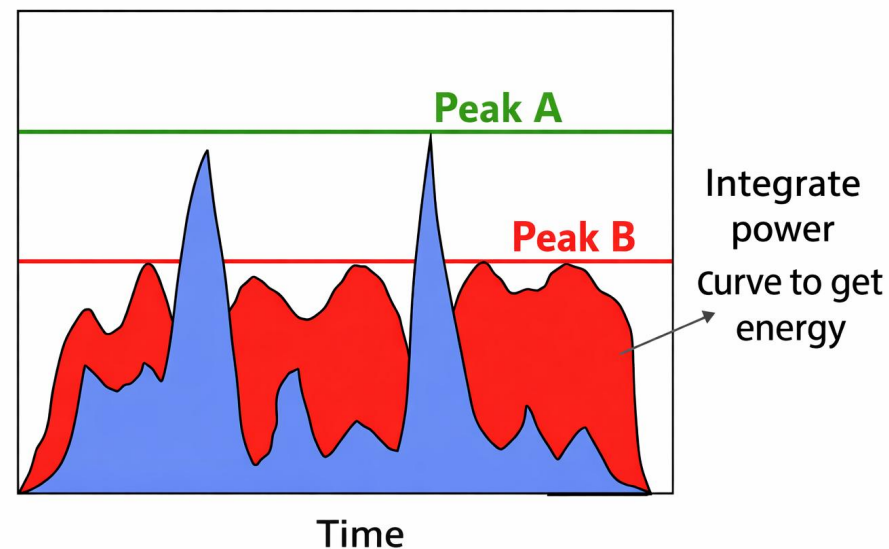
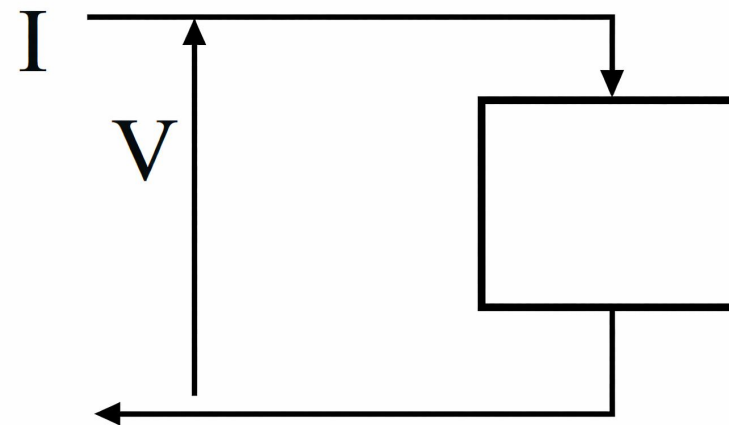
- 用于表征持续功耗
- 作为电源和冷却系统设计的目标
- 低于峰值功耗，但高于平均功耗

可以动态降低时钟频率来限制功耗。
单位任务能耗通常是更好的度量方式。



功耗与能量

- 能量
 - 以焦耳 (Joule) 计量
- 功率
 - 能量消耗的速率
 - [Watts = Joules/sec]
 - 瞬时功率 $P = V * I$
 - 即元件两端电压降与流经电流的乘积
- 示例
 - system A
 - 峰值功耗更高，但总能耗更低
 - system B
 - 峰值功耗更低，但总能耗更高



CMOS晶体管的功耗

动态功耗

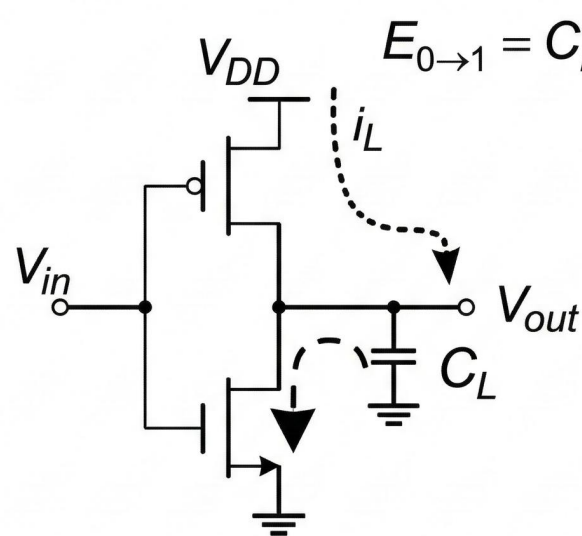
- 传统上主导整个功耗
- 在**晶体管工作**时产生功耗（即与数据相关）

静态功耗

- 随着晶体管尺寸不断缩小而变得越来越重要
- 来源于“**漏电流**”，即使没有开关活动也会流动
- 与芯片上的晶体管数量成正比

挑战

- 功耗已成为芯片设计的关键**限制因素**
 - 需要在芯片上将功耗均匀分配
 - 需要移除热量
 - 需要避免热点
 - 需要进行低功耗设计（如时钟门控、DVFS）



$$E_{0 \rightarrow 1} = C_L V_{DD}^2$$

$$E_R = \frac{1}{2} C_L V_{DD}^2$$

$$E_C = \frac{1}{2} C_L V_{DD}^2$$

□ Energy consumed in N cycles, E_N :

$$E_N = C_L \cdot V_{DD}^2 \cdot n_{0 \rightarrow 1}$$

$n_{0 \rightarrow 1}$ – number of 0→1 transitions in N cycles

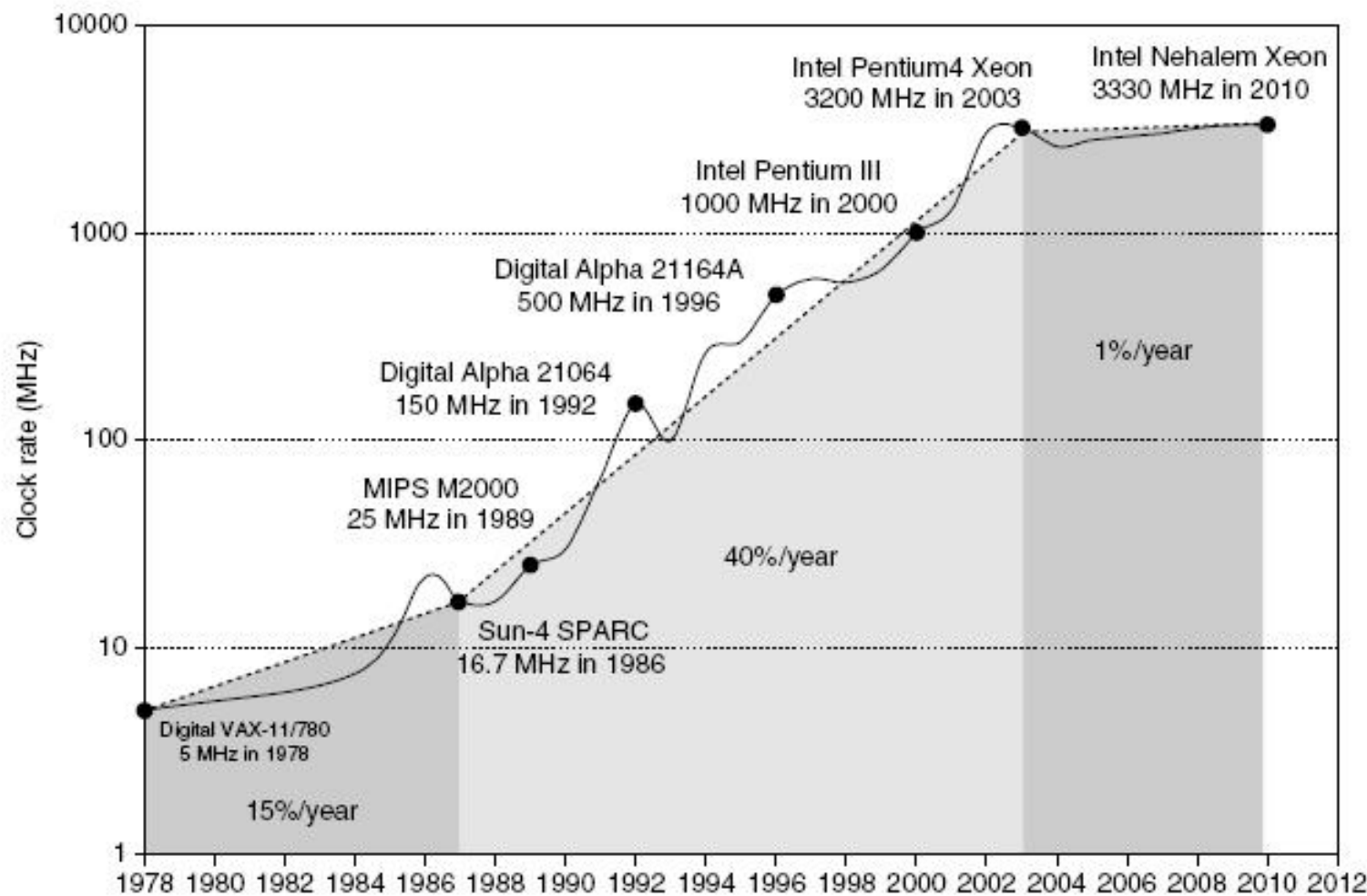
$$P_{avg} = \lim_{N \rightarrow \infty} \frac{E_N}{N} \cdot f = \left(\lim_{N \rightarrow \infty} \frac{n_{0 \rightarrow 1}}{N} \right) \cdot C_L \cdot V_{DD}^2 \cdot f$$

$$\alpha_{0 \rightarrow 1} = \lim_{N \rightarrow \infty} \frac{n_{0 \rightarrow 1}}{N} \cdot f$$

$$P_{avg} = \alpha_{0 \rightarrow 1} \cdot C_L \cdot V_{DD}^2 \cdot f$$

功耗

- Intel 80386 的功耗约为 2 W
- 3.3 GHz 的 Intel Core i7 功耗为 130 W
- 这些热量必须从一块 $1.5 \times 1.5 \text{ cm}$ 的芯片中散出。
- 这已经接近空气冷却所能达到的极限。



动态功耗

- 动态能耗
- 当晶体管从 $0 \rightarrow 1$ 或 $1 \rightarrow 0$ 切换时产生

$$\text{Energy}_{dynamic} \propto \frac{1}{2} \times \text{Capacitive load} \times \text{Voltage}^2$$

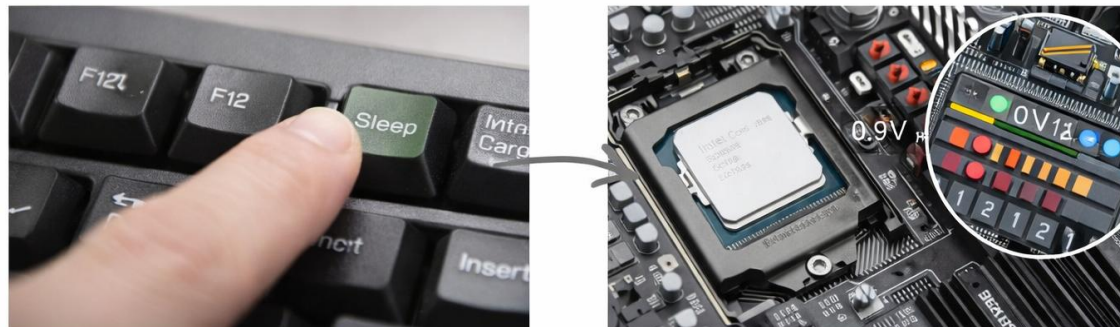
- 动态功耗

$$\text{Power}_{dynamic} \propto \frac{1}{2} \times \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency switched}$$

- 降低时钟频率能够降低功耗，但不能降低能耗。

降低动态功耗的技术

- 把空闲状态管理好
- 动态电压频率调整 (DVFS)
- 让 DRAM、磁盘进入低功耗状态
- 超频、关闭核心



动态电压频率调整 (Dynamic Voltage Frequency Scaling - DVFS)

- 示例: H&P5th P23

- DVFS 是一种低功耗设计技术, 正在现代处理器中广泛应用。

- 示例:

如果一个处理核心的电压和频率都降低 15%, 那么动态功耗会受到怎样的影响?

$$\text{Power Save} = \frac{P_{\text{new}}}{P_{\text{old}}} = \frac{C \times (V \times 0.85)^2 \times (F \times 0.85)}{C \times V^2 \times F} = 0.85^3 = 0.61$$

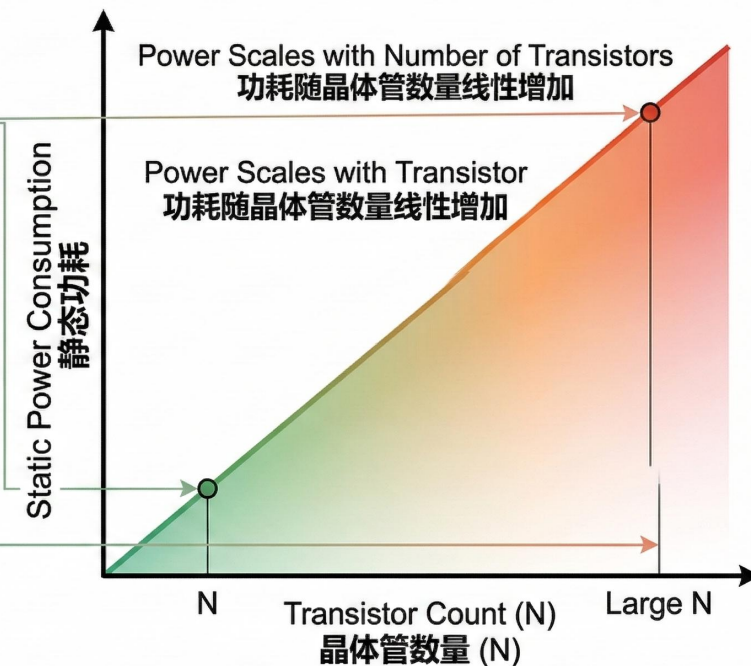
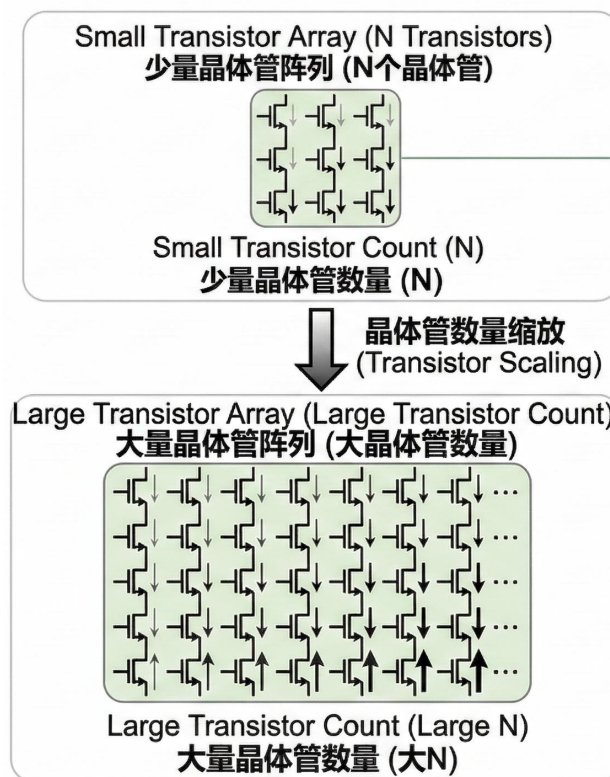
新状态下的功耗约为原来的 61%, 因此更加节能。

静态功耗定义

● 静态功耗

$$\text{Power}_{\text{static}} \propto \text{Current}_{\text{static}} \times \text{Voltage}$$

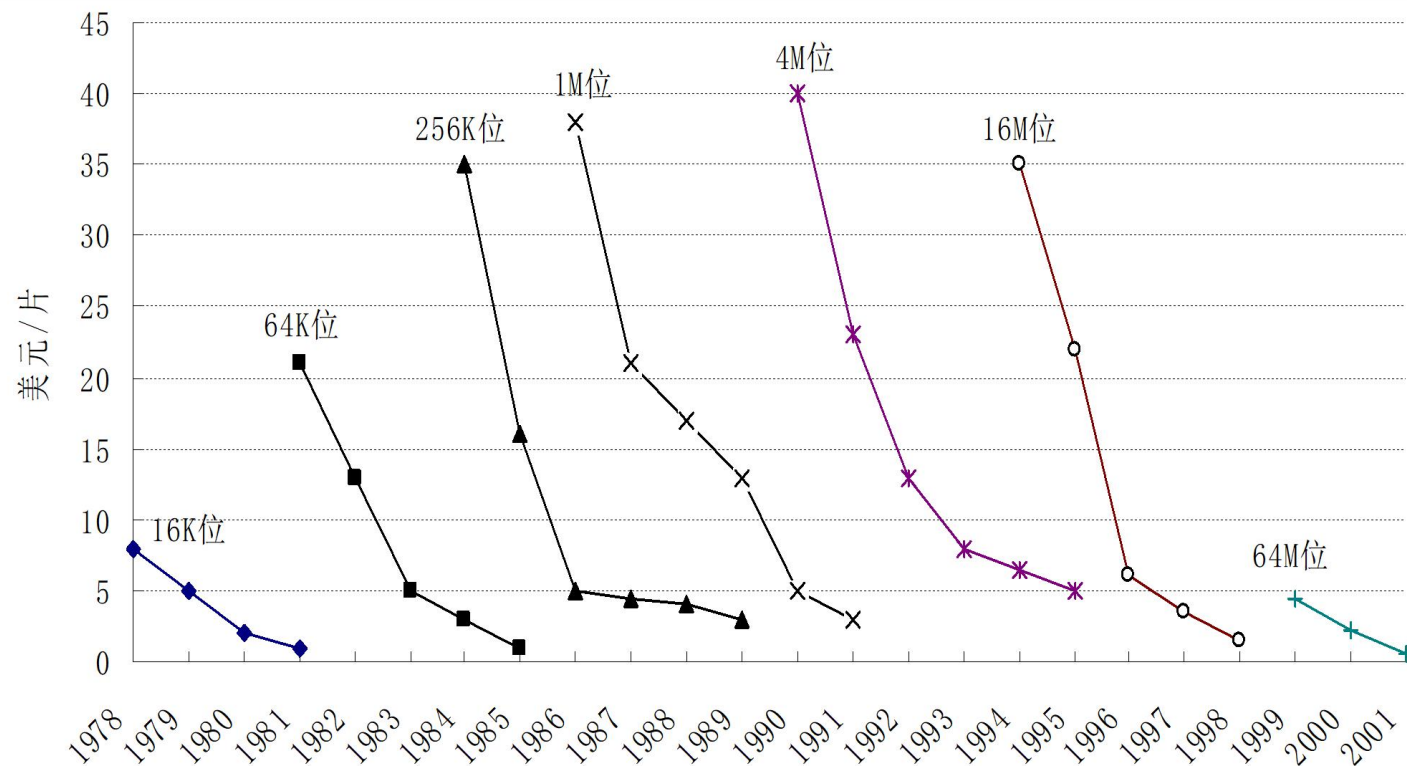
- 静态功耗会随着晶体管数量增加而增加。
- 降低静态功耗的方法：电源门控 (power gating)



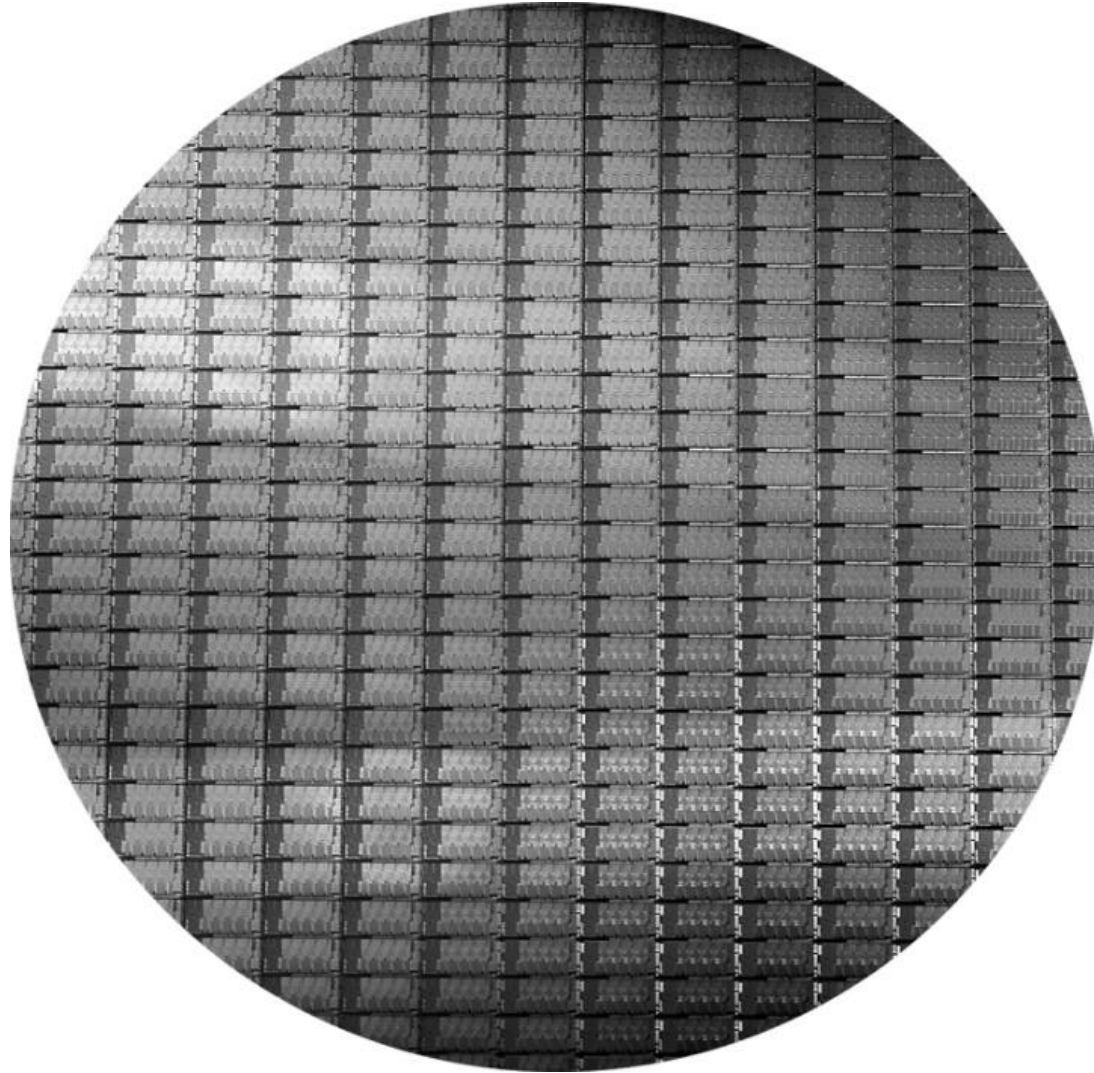
$$\text{Power}_{\text{static}} \propto \text{Current}_{\text{static}} \times V$$

成本变化趋势

- 成本会随着学习曲线而下降。
- DRAM: price **closely tracks** cost
- 微处理器的价格取决于出货量
 - 出货量每翻一倍，价格大约下降 10%。



芯片成本



This 300 mm wafer contains 280 full Sandy Bridge dies, each 20.7 by 10.5 mm in a 32 nm process. (Sandy Bridge is Intel's successor to Nehalem used in the Core i7.) At 216 mm², the formula for dies per wafer estimates 282. (Courtesy Intel.)

芯片成本

- 集成电路成本

$$\text{Cost of integrated circuit} = \frac{\text{Cost of die} + \text{Cost of testing die} + \text{Cost of packaging and final test}}{\text{Final test yield}}$$

$$\text{Cost of die} = \frac{\text{Cost of wafer}}{\text{Dies per wafer} \times \text{Die yield}}$$

$$\text{Dies per wafer} = \frac{\pi \times (\text{Wafer diameter}/2)^2}{\text{Die area}} - \frac{\pi \times \text{Wafer diameter}}{\sqrt{2} \times \text{Die area}}$$

玻色-爱因斯坦分布公式:

$$\text{Die yield} = \text{Wafer yield} \times 1 / (1 + \text{Defects per unit area} \times \text{Die area})^N$$

- 单位面积缺陷数在 2010 年约为每平方厘米 0.016–0.057 个缺陷。
- $N = \text{工艺复杂度因子} = 11.5\text{-}15.5$ (40 nm, 2010)

芯片成本—例题

- 例题: H&P 5th P31

Find the number of dies per 300 mm (30 cm) wafer for a die that is 1.5 cm on a side and for a die that is 1.0 cm on a side.

- ◆ 答案

When die area is 2.25 cm²:

$$\text{Dies per wafer} = \frac{\pi \times (30/2)^2}{2.25} - \frac{\pi \times 30}{\sqrt{2 \times 2.25}} = \frac{706.9}{2.25} - \frac{94.2}{2.12} = 270$$

Since the area of the larger die is 2.25 times bigger, there are roughly 2.25 as many smaller dies per wafer:

$$\text{Dies per wafer} = \frac{\pi \times (30/2)^2}{1.00} - \frac{\pi \times 30}{\sqrt{2 \times 1.00}} = \frac{706.9}{1.00} - \frac{94.2}{1.41} = 640$$

芯片成本—例题

- **例题: H&P 5th P31**

Find the die yield for dies that are 1.5 cm on a side and 1.0 cm on a side, assuming a defect density of 0.031 per cm^2 and N is 13.5.

- **答案**

The total die areas are 2.25 cm^2 and 1.00 cm^2 . For the larger die, the yield is

$$\text{Die yield} = 1 / (1 + 0.031 \times 2.25)^{13.5} = 0.40$$

For the smaller die, the yield is

$$\text{Die yield} = 1 / (1 + 0.031 \times 1.00)^{13.5} = 0.66$$

That is, less than half of all the large dies are good but two-thirds of the small dies are good.

性能的定义与度量

Airplane	Passenger Capacity	Cruising Range (miles)	Cruising Speed (m.p.h.)	Passenger Throughput (passenger x m.p.h)
Boeing 777	370	4630	610	225,700
Boeing 747	470	4150	610	286,700
Concorde	132	4000	1350	178,200
Douglas DC-8-50	146	8720	544	79,424

性能度量的两个基本维度

- 完成任务所需的时间
 - 执行时间、响应时间、总耗时、延迟
- 单位时间内完成的任务数
 - 执行速率、带宽、吞吐量

Airplane	DC to paris	Speed	Passengers	Throughput (passengers x mph)
Boeing 747	6.5 hours	610 mph	470	286,700
Concorde	3 hours	1350 mph	132	178,200

延迟 vs. 吞吐量

- 延迟 (Latency)
 - 完成一个任务所需要的“真实”时间
 - 当关注点是**单个任务时**非常重要
 - 正在使用单个应用程序的计算机用户
 - 实时嵌入式系统中的关键任务
- 吞吐量 (Throughput, 也称带宽 Bandwidth)
 - 单位时间内完成的任务数量
 - 一个不依赖具体任务总数的度量指标
 - 当关注点是**运行大量任务时**非常重要
- 大型数据处理中心的管理者更关心在给定时间内完成的总工作量

经典五级流水线

Instr. No.	Pipeline Stage						
1	IF	ID	EX	MEM	WB		
2		IF	ID	EX	MEM	WB	
3			IF	ID	EX	MEM	WB
4				IF	ID	EX	MEM
5					IF	ID	EX
Clock Cycle	1	2	3	4	5	6	7

- 流水线（Pipelining）
 - 能够提高指令吞吐量
 - 即单位时间内完成的指令数量
 - 但不会降低单条指令的执行时间
- 事实上，它通常还会略微增加单条指令的执行时间

性能度量

- Machine X 比 machine Y 快 n 倍

$$n = \frac{executionTime(Y)}{executionTime(X)} = \frac{performance(X)}{performance(Y)}$$

- 性能与执行时间互为倒数。**

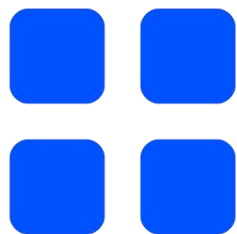
- 提高性能 → 性能值增大
- 改善执行时间 → 执行时间减小

- 比如**

- $executionTime(Y) = 4.8$, $executionTime(X) = 3.6$

- $n = 1.33$, i.e. X 比 Y 快了 33%

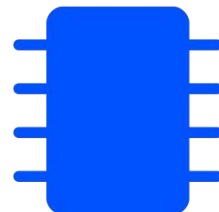
Benchmark Suites



真实软件应用

使用完整应用程序作为负载，贴近真实场景但耗时较长。

典型实例：GCC、SPEC CPU 2017



内核基准测试

提取核心计算片段，针对性强，便于精确测量硬件性能。

典型实例：Linpack



合成基准测试

人工构造程序模拟平均行为，执行速度快，适合快速评估。

典型实例：Whetstone & Dhrystone

Principle of Locality (局部性原理)

核心定义：程序执行时，数据和指令的访问倾向于集中在时间和空间上的一个局部区域。这是现代计算机体系结构（如缓存、虚拟内存）高效工作的根本原因。

主要类型 (Types)

时间局部性 (Temporal)：近期访问的数据很可能再次被访问（如循环变量）。

空间局部性 (Spatial)：访问某数据时，其邻近地址的数据也可能被访问（如遍历数组）。

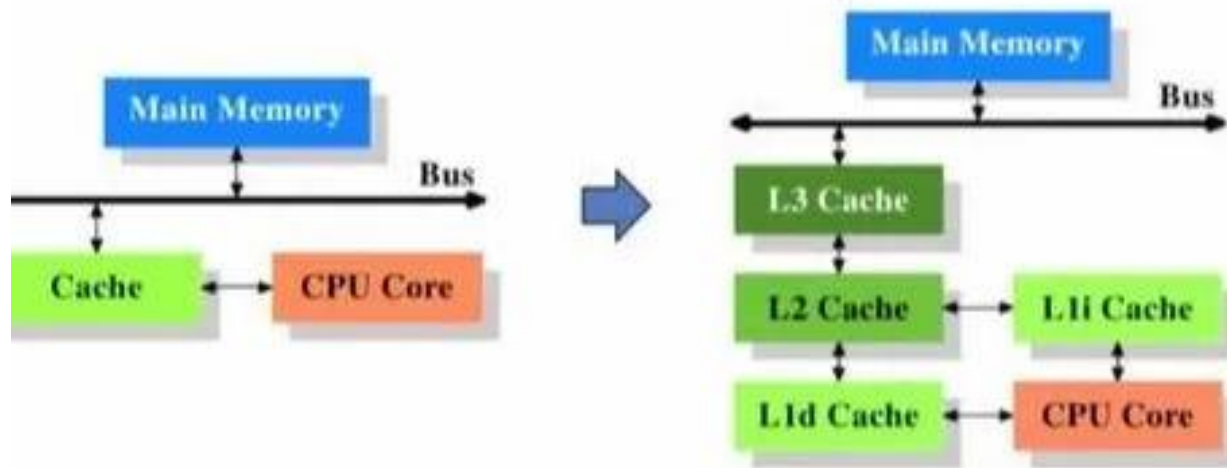
关键应用 (Applications)

CPU 缓存架构 (CPU Cache)

虚拟内存 (Virtual Memory)

预取技术与程序优化 (Prefetching & Optimization)

Principle of Locality - Cache Example



图示：CPU多级缓存架构与数据交互流程

时间局部性 (Temporal Locality)

核心机制是保留最近访问数据，避免重复读取主存。极大减少CPU等待时间。

空间局部性 (Spatial Locality)

加载整个缓存行而非单个字节。利用数组连续访问预取特性，最大化单次内存访问收益，降低访问频率。

性能影响 (Performance Impact)

命中率是核心指标。未命中惩罚极高，可能导致CPU暂停数百周期。。

Make the Common Case Fast

核心设计原则

决策依据

- ✓ 优先优化高频发生的“常见情况”

资源分配

- ✓ 将系统资源向高频任务倾斜

优化目标

- ✓ 针对系统平均行为进行优化

体系结构应用实例

CPU缓存机制

-  利用局部性原理，加速数据访问

分支预测技术

-  预测大概率执行路径，减少停顿

专用硬件单元

-  增加专用电路，降低关键路径延迟

阿姆达尔定律

$$\text{加速比} = 1 / [(1 - Fe) + Fe/Se]$$

-  **Fe (Fraction Enhanced)**
系统中可优化部分的占比

-  **Se (Speedup Enhanced)**
优化部分的加速倍数

阿姆达尔定律 (Amdahl's Law)

- What is the overall speedup after improving a component **x** of a system?



$$speedup = \frac{originalExecutionTime}{newExectionTime} = \frac{newPerformance}{originalPerformance}$$

- If component **x** is improved by **Sx** and component **x** affects a fraction **Fx** of the overall execution time then

$$speedup = \frac{1}{(1 - Fx) + Fx / Sx}$$

Fx / **Sx**: new exec. time of improved part

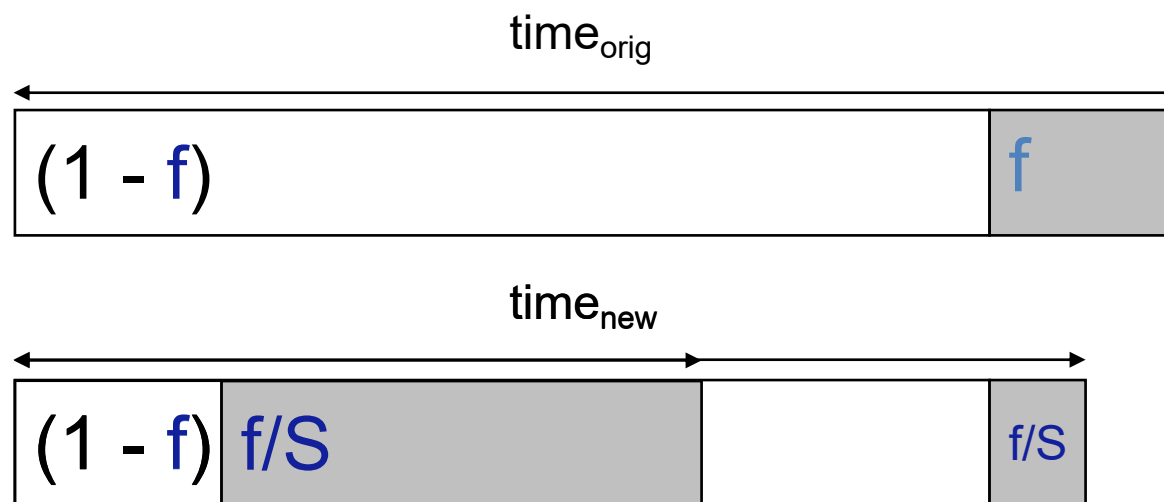
阿姆达尔定律 (Amdahl's Law)

$$\textit{Speedup} = \text{time}_{\text{without enhancement}} / \text{time}_{\text{with enhancement}}$$

An enhancement speeds up fraction f of a task by factor S

$$\text{time}_{\text{new}} = \text{time}_{\text{orig}} \cdot ((1-f) + f/S)$$

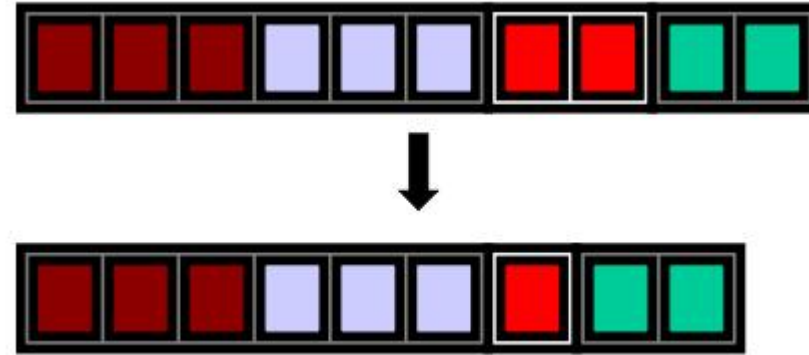
$$S_{\text{overall}} = 1 / ((1-f) + f/S)$$



阿姆达尔定律 (Amdahl's Law)

$$speedup = \frac{1}{(1 - Fx) + Fx / Sx}$$

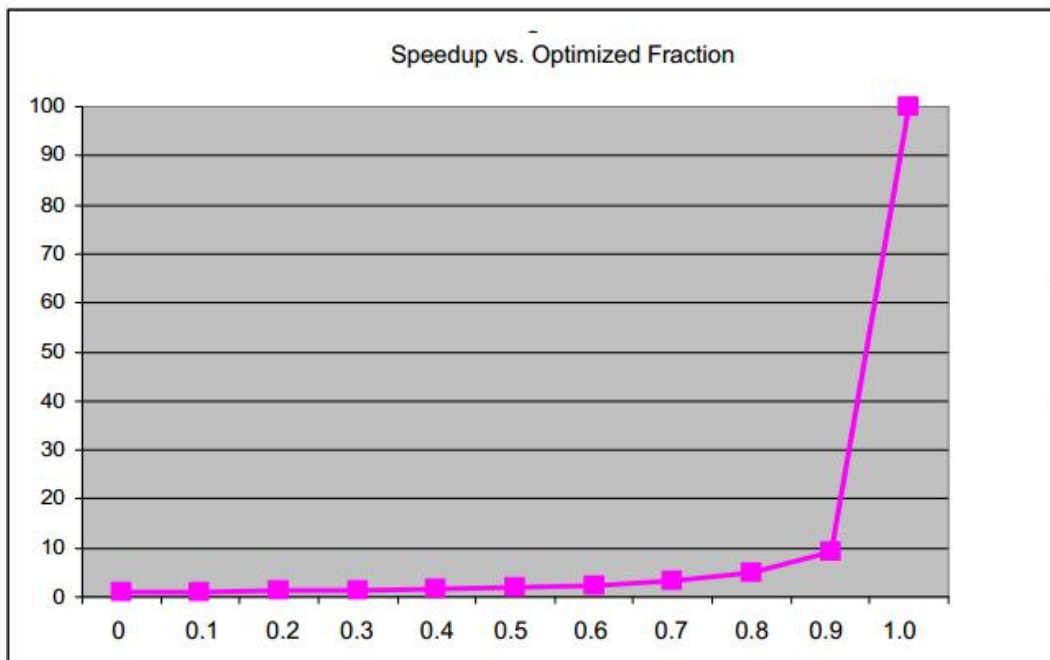
If we optimize the module for the floating-point instructions by **a factor of 2**, but the system will normally run programs with **only 20%** of floating point instructions then the speedup is only



$$speedup = \frac{1}{(1 - 0.2) + 0.2 / 2} = \frac{1}{0.9} = 1.111$$

阿姆达尔定律函数关系：加速比与优化比例的深度分析

阿姆达尔定律函数关系图 ($S_x=100$)



图表趋势解读

低 F_x 区域 ($F_x < 0.5$)

↓ 增长极其缓慢，优化效果不明显，接近线性增长。

中 F_x 区域 ($0.5 \leq F_x < 0.9$)

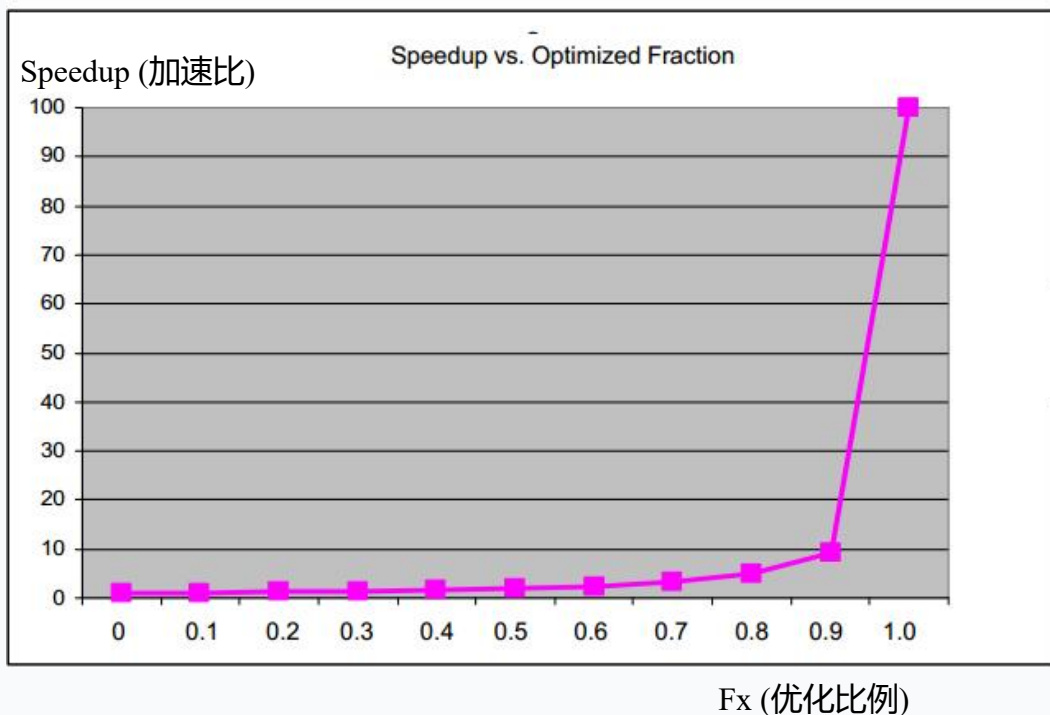
↑ 加速比开始逐渐增长，但增长速度仍相对平缓。

高 F_x 区域 ($F_x \geq 0.9$)

↗ 加速比急剧增长，此时优化能带来显著的性能提升。

阿姆达尔定律与收益递减法则：优化的边际效益分析

阿姆达尔定律函数关系图 ($S_x=100$)



随着优化倍数(S_x)增加，加速比提升趋缓，上限受限于被优化部分比例(F_x)

边际效益分析

收益递减法则：

- 连续投入等量资源，边际产出逐渐下降。

阿姆达尔定律印证：

- 公式 $\text{Speedup} = 1 / [(1-F_x) + F_x/S_x]$
- 量化了非线性关系， S_x 增大时， F_x/S_x 项对整体贡献减弱。

阿姆达尔定律应用案例：Web服务器处理器升级的加速比分析

案例背景：Web服务器升级

- **场景：**Web服务器性能优化，计算耗时占40%，I/O等待占60%。
- **措施：**升级处理器，计算速度提升10倍 ($S_x=10$)。
- **目标：**提升服务器整体响应速度与吞吐量。

计算过程详解 (Amdahl's Law)

1. 不可优化部分(I/O): $1 - F_x = 0.6$
2. 优化后计算耗时: $F_x / S_x = 0.4 / 10 = 0.04$
3. 总耗时比例: $0.6 + 0.04 = 0.64$
4. 最终加速比: $1 / 0.64 = 1.56$

CPU Time：程序执行时间的核心指标

⚙️ 核心计算公式

$$\begin{aligned}\text{CPU Time} &= (\text{Clock Cycles}) \times (\text{Cycle Time}) \\ &= (\text{Clock Cycles}) / (\text{Clock Frequency})\end{aligned}$$

程序的CPU执行时间取决于两个关键因素：

- 程序所需的时钟周期数 (Cycles)
- 单个时钟周期的时间 (Cycle Time)

🕒 CPU时间的构成

👤 用户CPU时间 (User Time)

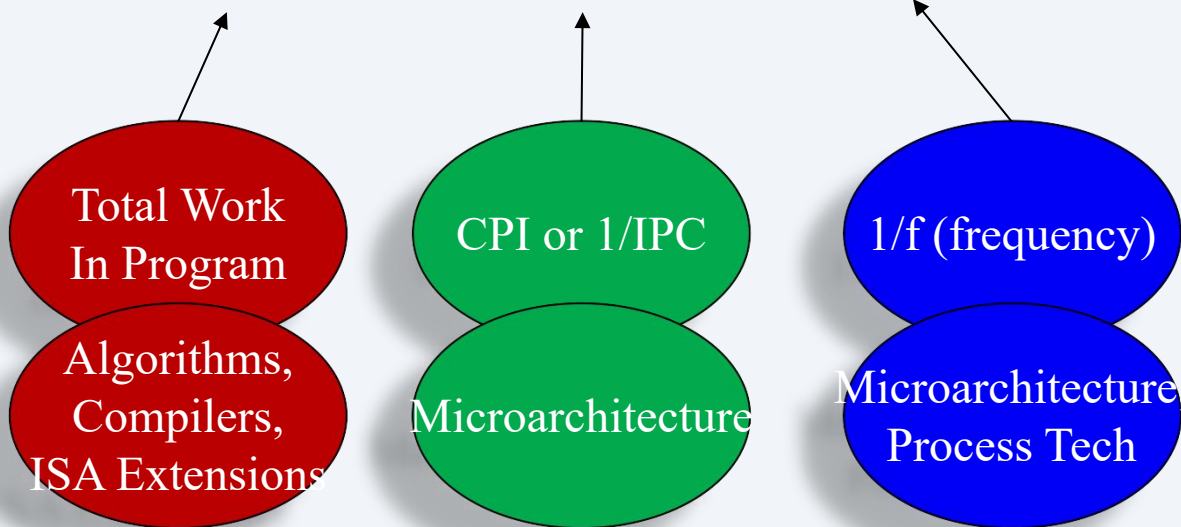
程序在用户态执行指令所消耗的时间，是性能评估的核心。

📊 系统CPU时间 (System Time)

操作系统为程序执行任务（如I/O）所消耗的时间。

处理器性能铁律：程序执行时间的三大决定因素

$$\frac{\text{Time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Time}}{\text{Cycle}}$$



核心公式：程序执行时间 = 指令数 × CPI × 时钟周期

程序执行时间由**指令数**、**CPI**和**时钟周期**共同决定。优化性能需从这三个维度入手，而非单一追求频率。

处理器性能铁律：各层级的影响分析

各层级对CPU时间三要素的影响

层级 (Layer)	IC	CPI	CCT
程序 (Program)	✓		
编译器 (Compiler)	✓		
指令集 (ISA)	✓	✓	
硬件组织 (HW Org)		✓	✓
硬件技术 (HW Tech)			✓

CPU Time = IC × CPI × CCT

💡 核心思想：分层影响机制

程序与编译器：主要决定指令数 (IC)，是优化的第一关口。

指令集架构 (ISA)：同时影响 IC 和 CPI，是架构设计核心。

硬件组织：主要影响 CPI 和 CCT，决定执行效率。

硬件技术：比如制程工艺，直接决定时钟周期 (CCT)，是性能的物理基石。

CPU Time：三大核心影响因素

核心公式与因素解析

$$\text{CPU Time} = \text{IC} \times \text{CPI} \times \text{CCT}$$

📄 IC (Instruction Count)指令数

指程序执行所需的总指令数量。

🔌 CPI (Clock Cycles per Instruction)每指令周期数

执行一条指令平均所需的时钟周期数

🕒 CCT (Clock Cycle Time)时钟周期时间

单个时钟周期的时间长度

性能优化启示与要点

三大因素的等同重要性

CPU时间由三者共同决定，且影响是等同的。在任何一个因素上实现10%的改进，都能带来CPU时间10%的降低。

通过并行性提升性能：多层次优化策略

系统级 (System Level)

优化手段：使用多处理器、多磁盘等。

核心思想：增加资源数量并行处理。

微处理器级 (Microprocessor)

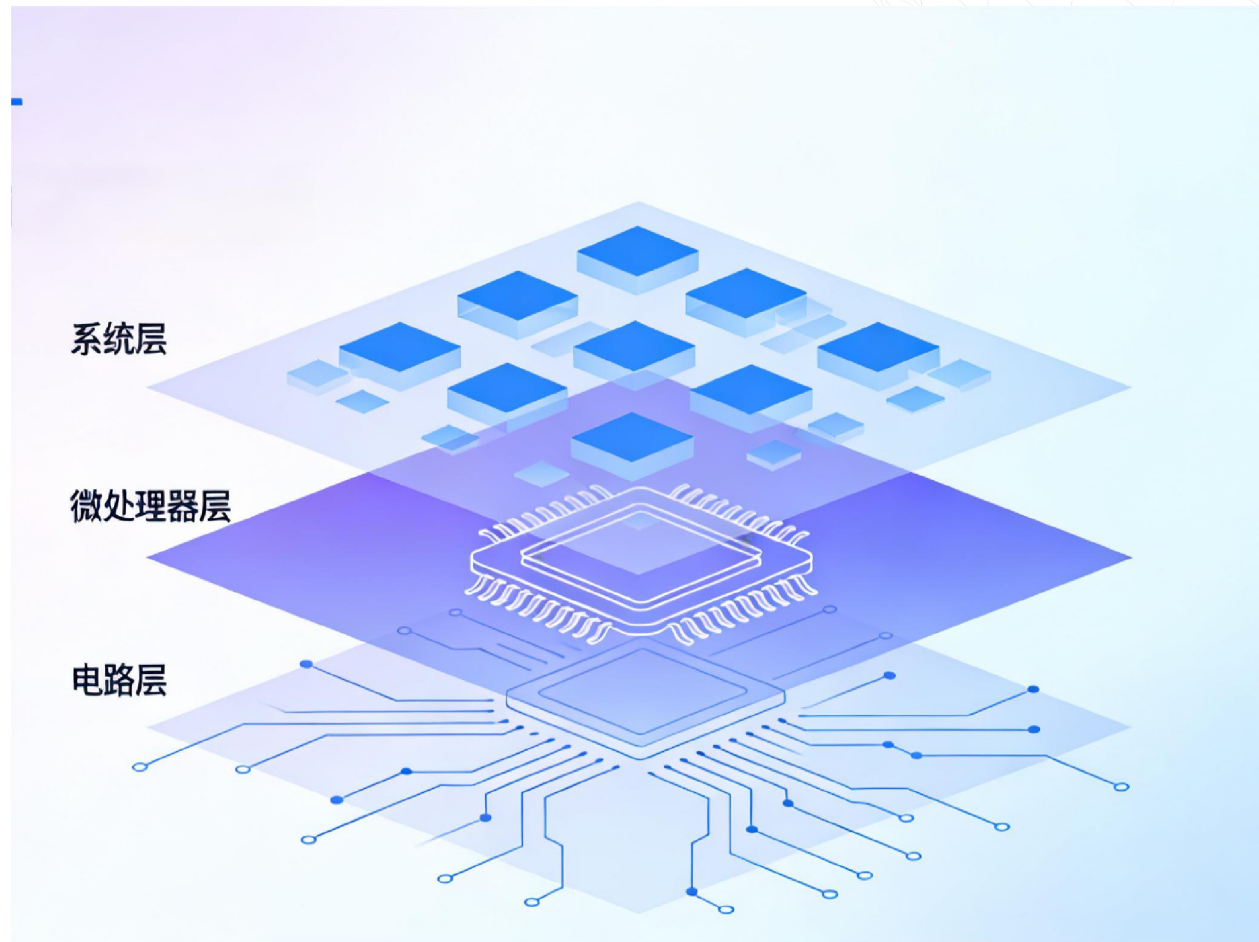
优化手段：开发指令级并行。

核心思想：时间重叠或空间并行执行指令。

电路级 (Circuit Level)

优化手段：采用并行电路设计。

核心思想：将串行操作改为并行操作。



CPU Time深度解析：按指令类型的分解

核心公式分解

1. CPU Time 总时间分解

$$\text{CPU Time} = (\sum (\text{IC}_i \times \text{CPI}_i)) \times \text{CCT}$$

2. CPI 加权平均分解

$$\text{CPI} = \sum (\text{IF}_i \times \text{CPI}_i)$$

结论

指令频率的影响

高频指令（高 IF_i ）对总CPI贡献最大。优化高频指令的CPI能最有效地降低总CPU时间。

CPI的实际测量

实际CPI必须包含流水线、缓存等微架构特性的影响，不能仅从手册计算。

CPU Time案例分析：性能优化方案比较

- **Example: H&P 5th P50**

Suppose we have made the following measurements:

Frequency of FP operations = 25%

Average CPI of FP operations = 4.0

Average CPI of other instructions = 1.33

Frequency of FPSQR = 2%

CPI of FPSQR = 20

Assume that the two design alternatives are to decrease the CPI of FPSQR to 2 or to decrease the average CPI of all FP operations to 2.5. Compare these two design alternatives using the processor performance equation.

CPI案例分析：不同指令序列的性能比较

- Alternative compiled code sequences using instructions in classes A, B, C. **What is avg. CPI?**

Class	A	B	C
CPI for class	1	2	3
IC in sequence 1	2	1	2
IC in sequence 2	4	1	1

- Sequence 1: IC = 5
 - Clock Cycles
 $= 2 \times 1 + 1 \times 2 + 2 \times 3$
 $= 10$
 - Avg. CPI = $10/5 = 2.0$
- Sequence 2: IC = 6
 - Clock Cycles
 $= 4 \times 1 + 1 \times 2 + 1 \times 3$
 $= 9$
 - Avg. CPI = $9/6 = 1.5$

案例分析：平均指令执行时间计算

案例背景与数据概览

场景：非流水线处理器，时钟周期时间 (CCT) = 2ns

操作类型	频率 (IFi)	CPIi	IFi × CPIi
ALU运算	0.5	4	2.0
Load加载	0.2	5	1.0
Store存储	0.1	5	0.5
Branch分支	0.2	4	0.8

计算过程与结果分析

步骤 1：计算平均 CPI

公式： $CPI = \sum (IF_i \times CPI_i)$

计算： $2.0 + 1.0 + 0.5 + 0.8 = 4.3$

步骤 2：计算平均指令执行时间

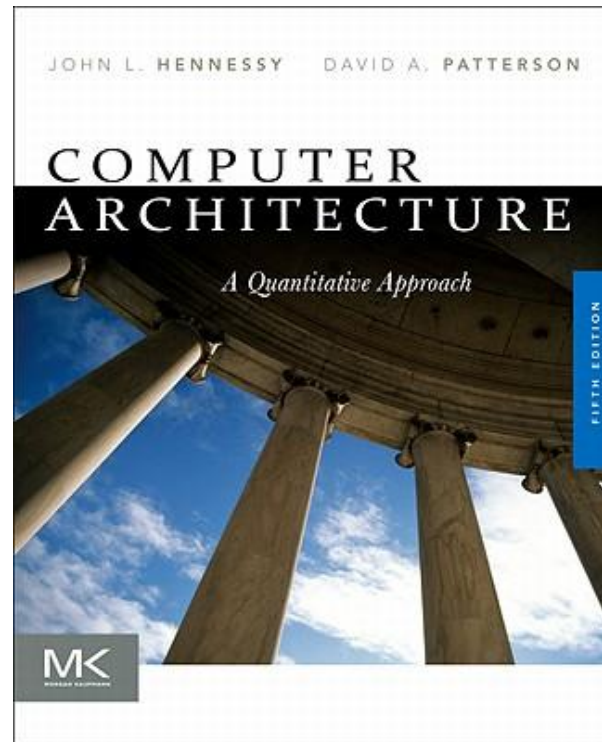
公式：平均时间 = CPI × 时钟周期 (2ns)

计算： $4.3 \times 2ns = 8.6ns$

结论：该处理器平均每条指令的执行时间为**8.6纳秒**。

参考教材

- Computer Architecture: A Quantitative Approach, 5th ed.," John L. Hennessey and David A. Patterson, Morgan Kaufman, 2011
- Additional readings will be assigned in class:
 - journal and conference papers
 - excerpts from other books
 - technical manuals



Homework #1

H&P 5th

Page 67: 1.15 and 1.16

Due 2026/04/07